Predicting Household Appliance Energy with Neural Networks and Piecewise-Linear Inputs

Bayu Febriansyah
University of Illinois Urbana-Champaign, Urbana, Illinois 61801

This project compares ten neural-network configurations for estimating household appliance energy during a measured ten-minute interval. The reference study's original training and test partitions are retained. Five configurations vary network size and training settings; five add quantile-based piecewise-linear features. The selected 256-bin network has the lowest mean validation root-mean-square error (RMSE), 70.75 Wh. Its test RMSE is 64.05 Wh, mean absolute error is 28.37 Wh, and coefficient of determination is 0.603. The reference gradient-boosting model reports 66.65 Wh, 35.22 Wh, and 0.57. Increasing to 512 bins lowers test RMSE to 63.45 Wh but raises mean validation error and adds 66.5% more parameters. These results support the selected network as a candidate for this benchmark.

I. Introduction

Can a feedforward neural network match the published models for the appliance-energy dataset studied by Candanedo et al. [1]? This project examines that question using the same reference training and test partitions. The target is the total appliance energy used during a ten-minute measurement interval, in watt-hours (Wh). Inputs include measurements from that interval, so the task is contemporaneous estimation rather than forecasting a future interval.

Candanedo et al. [1] compared multiple linear regression (MLR), radial-kernel support-vector regression (SVM), random forest (RF), and gradient boosting (GBM). Their reported GBM test RMSE of 66.65 Wh provides the main numerical benchmark. Here the predictor is a multilayer perceptron trained by backpropagation [2, 3].

The comparison contains ten configurations. Runs 1–5 examine network width, depth, and training settings. Runs 6–10 add piecewise-linear numerical features and increase their resolution. The run numbers organize the configurations by complexity. The aim is to identify a useful configuration within this set, explain the observed error changes, and assess whether the final increase in size is justified.

II. Background

A. Reference study and dataset

Candanedo et al. [1] collected 19,735 ten-minute observations from a low-energy house in Belgium between January 11 and May 27, 2016. The reference partition contains 14,803 training rows and 4,932 test rows. The original files are distributed through the authors' repository [4], with dataset documentation at the UCI Machine Learning Repository [5]. This project retains that partition. The code checks the expected file hashes and row counts before fitting a model; the dataset is downloaded separately rather than included in the submission.

The reference GBM fits regression trees sequentially to reduce prediction error [6, Chap. 10]. Candanedo et al. [1] report 10,900 trees, maximum depth five, and ten-fold cross-validation repeated three times. Their Table 5 gives test RMSE of 66.65 Wh, MAE of 35.22 Wh, and $R^2 = 0.57$. These are published comparison values; a GBM is not retrained in this project.

B. Inputs and measurement limits

The network uses 25 measurements: lighting energy, nine temperature/relative-humidity pairs, and six airport-weather variables. The weather variables are outdoor temperature, pressure, relative humidity, wind speed, visibility, and dew point. Sensor pair six measures outdoor conditions near the house [1, 5]. Time is represented by seconds since midnight (NSM), day of week, and weekday/weekend status. One-hot encoding of the two calendar categories gives nine indicators, for 35 original input columns [7]. Artificial variables `rv1` and `rv2`, raw dates, and previous target values are excluded.

Temperatures are in °C, relative humidity in percent, pressure in mmHg, wind speed in m/s, visibility in km, lighting energy in Wh, and NSM in seconds. The target ranges from 10 to 1,080 Wh in increments of 10 Wh in the recorded study. The implementation uses every row in each reference file, without imputation or exclusion. Before fitting, it checks required values for missing or nonfinite entries and verifies timestamp uniqueness and training/test disjointness.

The reference study discusses sensor placement, a single house, limited seasonal coverage, and weather measurements from a station about 12 km away [1, Secs. 2–3.6]. It reports DHT22 accuracy of about ±0.5 °C and ±3 humidity percentage points. Hourly weather was interpolated to ten-minute intervals. Lighting is measured during the target interval, and weather interpolation can involve the next hourly reading.

III. Technical Approach

A. Standard neural network

1. Preparing the inputs and defining the model

Let $\mathbf{x}_i$ be the input vector and y_i the appliance energy in Wh. Each input and the target are standardized by subtracting the fitting-data mean and dividing by its population standard deviation [8]:

$$\mathbf{z}_i = (\mathbf{x}_i - \boldsymbol{\mu}_x) \oslash \boldsymbol{s}_x, \qquad u_i = (y_i - \mu_y)/s_y. \tag{1}$$

Here $\oslash$ denotes entrywise division. Scaling places inputs with different units on similar numerical scales [9, Sec. 4.3]. Means, standard deviations, and category levels use fitting rows only, preventing preprocessing leakage [10]. Calendar indicators are also standardized. `OneHotEncoder` uses `handle_unknown='ignore'` for unseen categories [7].

The model is a multilayer perceptron (MLP). Each layer multiplies its inputs by weights, adds biases, and applies an activation function [2, Chap. 6]. With L hidden layers,

$$\mathbf{h}^{(0)} = \mathbf{z}, \quad \mathbf{a}^{(\ell)} = \mathbf{W}_\ell \mathbf{h}^{(\ell-1)} + \mathbf{b}_\ell, \quad \mathbf{h}^{(\ell)} = \max(\mathbf{0}, \mathbf{a}^{(\ell)}), \quad g_\theta(\mathbf{z}) = \mathbf{w}_o^T \mathbf{h}^{(L)} + b_o. \tag{2}$$

Here θ contains all weights and biases. ReLU returns $\max(0, a)$ and introduces nonlinearity [11]; without nonlinear activations, stacked affine layers remain affine [2, Secs. 6.1–6.3]. The output layer is linear for the continuous regression target [2, Sec. 6.2.2.1].

The basic model has one hidden layer with 64 units. Increasing width adds units; increasing depth adds layers [2, Sec. 6.4]. Both increase the model's capacity, but neither is guaranteed to improve validation error. For layer sizes $d_0, \ldots, d_L, d_{L+1} = 1$, counting the weights and biases gives $\sum_{\ell=1}^{L+1}(d_{\ell-1} + 1)d_\ell$ parameters. The 35–64–1 model has 2,369 parameters. A 35–256–128–1 model has 42,241.

2. Loss function and training

The network is trained on batches of observations using mean squared error (MSE). Weight decay adds an L_2 penalty [2, Secs. 6.2.1, 7.1.1, 8.1.3]:

$$\mathcal{L}_B = \frac{1}{|B|}\sum_{i \in B}(g_\theta(\mathbf{z}_i) - u_i)^2, \qquad \mathcal{J}_B = \mathcal{L}_B + \frac{\lambda}{2}\|\theta\|_2^2. \tag{3}$$

In this equation, B is a batch and λ sets the penalty strength. Backpropagation uses the chain rule to calculate how each weight affects the loss [3]. The output derivative is $2(g_\theta - u_i)/|B|$. ReLU uses an activation gradient of $\mathbb{1}(a > 0)$, choosing zero at its kink [2, Sec. 6.3.1]. Squaring the errors gives large errors more weight, which agrees with using RMSE as the main metric [12]. The target is standardized during training and converted back to Wh for evaluation.

Adam adjusts the updates using running averages of gradients and squared gradients [13]. The PyTorch implementation used here applies weight decay to the gradient before these averages [14]:

$$\widetilde{\mathbf{g}}_t = \nabla_\theta \mathcal{L}_B + \lambda\theta_t, \quad \mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\widetilde{\mathbf{g}}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\widetilde{\mathbf{g}}_t^2, \quad \theta_{t+1} = \theta_t - \eta_t \frac{\widehat{\mathbf{m}}_t}{\sqrt{\widehat{\mathbf{v}}_t} + \epsilon}. \tag{4}$$

Here t counts optimizer updates and η_t is the learning rate. Bias correction gives $\widehat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t)$ and $\widehat{\mathbf{v}}_t = \mathbf{v}_t/(1-\beta_2^t)$ [13]. Squares and divisions are applied to each vector entry. The default values are $\beta_1 = 0.9$, $\beta_2 = 0.999$,

and $\epsilon = 10^{-8}$. This is coupled weight decay in Adam, not AdamW [14]. The penalty includes biases and uses $\lambda = 0$ or 0.001.

All runs start at learning rate 0.001, use batches of 256, and allow at most 240 epochs. An epoch is one pass through the fitting data. Early stopping keeps the best validation checkpoint [2, Sec. 7.8]. In this study, an improvement must exceed 0.01 Wh, and training stops after 25 epochs without that improvement. Runs 5–10 also use `ReduceLROnPlateau` [15]: factor 0.5, patience eight, and a minimum rate of 10^{-5}. Its relative threshold is the default 10^{-4}; this is separate from the 0.01 Wh checkpoint rule.

Runs 1–4 use 64, 128, 128/64, and 256/128 hidden units, with no decay and a fixed rate. Run 5 keeps 256/128 units and adds decay and scheduling.

3. Validation and performance measures

Fitting, model selection, and final testing have different roles [6, Chap. 7]. First, 80% of the author training file is used to fit the model and 20% to choose the number of epochs. This initial split selects the training schedule. It has 11,842 fitting rows and 2,961 validation rows. The code uses `train_test_split` [16], seed 20261025, and strata formed from five target-quantile groups.

The selected epoch count and learning rate at each epoch are then fixed. The model is retrained on three more 80/20 splits, with seeds 20261030, 20261031, and 20261032. Each fit calculates its own scaling and later its own piecewise knots. These three validation sets do not change the training length or learning-rate schedule. Finally, the model is trained on all 14,803 author training rows and tested on the 4,932 author test rows. Keeping the same epoch count keeps the number of passes fixed, although a larger training set requires more batch updates.

All ten configurations use initialization seed 42 and batch-order seed 1042. Mean validation RMSE is the arithmetic mean of the three additional split scores; it excludes the schedule-selection split. Each configuration therefore requires five fits. The final configuration is selected by the lowest mean validation RMSE among the ten candidates.

The additional validation partitions come from the same 14,803-row pool used to select the schedule. Their evaluation rows can therefore have influenced that schedule through the initial fit or its validation loss. Fitting transformations separately avoids preprocessing leakage, but does not make these scores an independent evaluation of the full schedule-selection procedure. They are used for descriptive model comparison. A nested procedure would choose the schedule using only the fitting subset of each outer partition. The test set was also inspected during development; this limits the strength of the final benchmark comparison [17].

Predictions are returned to Wh and clipped below zero: $\widehat{y} = \max(0, \mu_y + s_y g_\theta)$. No upper cap or rounding is used. For n evaluation observations, the standard metrics are [12]

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \widehat{y}_i)^2}, \qquad \mathrm{MAE} = \frac{1}{n}\sum_i |y_i - \widehat{y}_i|, \qquad R^2 = 1 - \frac{\sum_i (y_i - \widehat{y}_i)^2}{\sum_i (y_i - \overline{y}_{\mathrm{eval}})^2}. \tag{5}$$

RMSE and MAE are in Wh. R^2 is dimensionless and uses the evaluation-target mean. The reported epoch counts for Runs 1–10 are 105, 178, 158, 93, 105, 81, 74, 97, 96, and 95. Refits replay the learning rates through the selected epoch. Two CPU threads and deterministic Torch operations are requested [18].

B. Adding piecewise-linear input features

1. Piecewise feature construction

A ReLU network already describes a piecewise-linear function [2, Sec. 6.3.1]. The input expansion supplies fixed breakpoints in each numerical input before the hidden layers. This follows the idea of a basis expansion: transform inputs into additional functions that the model can combine [6, Chap. 5]. It may help the NN learn changes in the response over different input ranges. Numerical input embeddings are an established approach in tabular NN modeling [19]; this project adapts that idea to the appliance dataset.

The method is applied to 26 numerical inputs: the 25 measurements and NSM. Following Gorishniy et al. [19, Sec. 3.2.1], quantiles at probabilities k/B, for $k = 0, \ldots, B$, define the bin edges. Here B is the requested number of bins. Only fitting rows are used, and repeated edges are removed. For adjacent distinct edges $q_{j,k}$ and $q_{j,k+1}$ of input z_j, the clipped ramp is

$$\psi_{j,k}(z_j) = \min\left(1, \max\left(0, \frac{z_j - q_{j,k}}{q_{j,k+1} - q_{j,k}}\right)\right), \qquad k = 0, \ldots, K_j - 1. \tag{6}$$

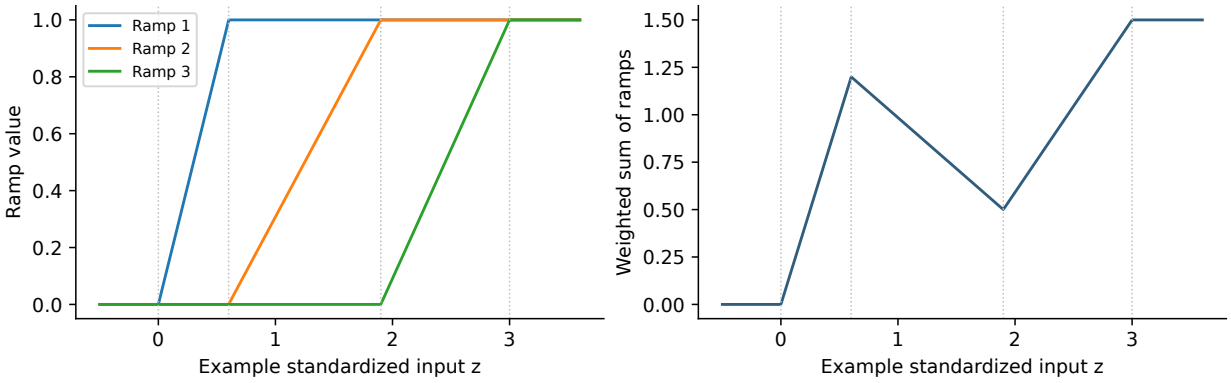


Fig. 1 Example of clipped ramps and their weighted sum, based on the encoding idea in Gorishniy et al. [19]. This is an original illustration, not a fitted appliance response.

This adapts the encoding in Ref. [19, Sec. 3.2, Eq. 1]. There are K_j nonzero-width intervals for input j. Each ramp is zero below its interval, increases linearly inside it, and stays one above it. Combining ramps lets the slope change at the bin edges while keeping the function continuous (Fig. 1). Quantiles place edges according to the observed input distribution [19, Sec. 3.2.1], so bin widths need not be equal.

2. Adding the features to the network

Each ramp is standardized using its fitting-data mean and standard deviation [8]. The ramps are then added to the original 35 input columns:

$$\tilde{\mathbf{z}} = \left[\mathbf{z}, \{(\psi_{j,k} - \mu_{j,k})/s_{j,k}\}_{j,k}\right], \qquad \widehat{y} = \max(0, \mu_y + s_y g_\theta(\tilde{\mathbf{z}})). \tag{7}$$

The nine calendar indicators are kept but are not expanded. The hidden layers stay at 256/128 units, with the same optimizer, loss, decay, and scheduling rules as Run 5. Only the NN weights and biases are learned. The bin edges and ramp scaling stay fixed during training.

More input columns mean more weights in the first layer. With D inputs, the network has $256D + 33{,}281$ parameters. The representation and parameter count therefore change together. No standard NN matched to the expanded networks in parameter count is included.

Runs 6–10 request 8, 32, 128, 256, and 512 bins. Quantiles use NumPy's linear interpolation [20], and the final input arrays use float32. Repeated values reduce the actual number of features. The full-training 256-bin model has 4,996 inputs and 1,312,257 parameters. The 512-bin model has 8,407 inputs and 2,185,473 parameters. These counts can vary slightly across fitting subsets because their quantiles differ.

This implementation keeps the raw inputs and clips every added ramp. Gorishniy et al. [19, Sec. 3.2] replace the scalar inputs and allow the end intervals to extrapolate. Here the added ramps stop changing outside the fitting range, but the retained raw inputs can still extrapolate. The model therefore has no guarantee of physically correct behavior outside the observed range.

IV. Results

A. Standard network comparisons

Table 1 reports all ten configurations, and Fig. 2 summarizes their errors and sizes. Parameter counts refer to the final fit on all reference training rows. Tables show two decimal places for errors and three for R^2. Approximate differences below use the archived summary values, which retain three decimals for errors; fresh runs export metrics without display rounding.

Increasing a single hidden layer from 64 to 128 units lowers mean validation RMSE from 82.40 to 81.11 Wh. Adding a second layer and then widening both layers lowers it further, to 77.76 and 76.06 Wh. Test error follows the same ordering. Larger configurations therefore perform better within this comparison. That pattern is consistent with a capacity limitation in the smaller networks, but the experiments do not isolate capacity from optimization effects.

Table 1 Ten NN configurations. Errors are in Wh. Mean validation RMSE averages three additional 80/20 partitions. All models use initialization seed 42. Runs 6–10 retain the Run 5 hidden layers and training rules.

Run	Configuration	Mean val. RMSE	Test RMSE	Test MAE	Test R^2	Parameters
1	64 units	82.40	82.22	45.15	0.345	2,369
2	128 units	81.11	79.81	44.17	0.383	4,737
3	128/64 units	77.76	75.12	37.78	0.453	12,929
4	256/128 units	76.06	73.17	36.08	0.481	42,241
5	256/128, decay/LR	75.30	73.01	35.13	0.484	42,241
6	8-bin inputs	72.40	66.79	30.32	0.568	93,441
7	32-bin inputs	71.79	65.66	29.44	0.582	241,665
8	128-bin inputs	71.27	64.57	28.99	0.596	764,673
9	256-bin inputs	70.75	64.05	28.37	0.603	1,312,257
10	512-bin inputs	71.38	63.45	28.65	0.610	2,185,473

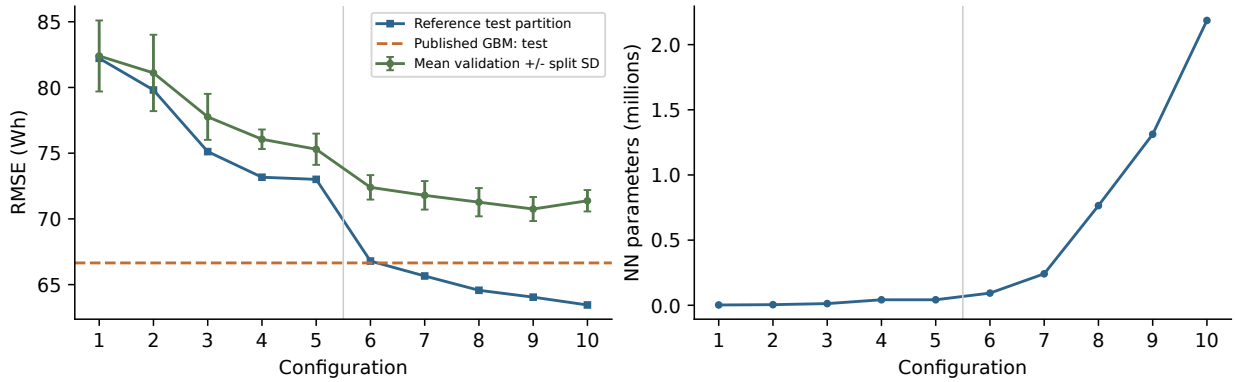


Fig. 2 Errors and parameter counts. Error bars are sample standard deviations ($n-1$ denominator) across three overlapping validation partitions, not confidence intervals. The published GBM line applies only to test scores.

Run 5 retains the 256/128-unit architecture and adds weight decay and learning-rate reduction together. Mean validation RMSE falls by about 0.76 Wh and test RMSE by 0.16 Wh. The small improvement belongs to their combined use; it does not identify which training change helped. Run 5 still has test RMSE of 73.01 Wh, about 6.36 Wh above the reference GBM. This remaining gap provides a reason to examine an input expansion alongside the standard network.

B. Piecewise inputs and final selection

Adding eight-bin ramps in Run 6 lowers mean validation RMSE from 75.30 to 72.40 Wh and test RMSE from 73.01 to 66.79 Wh. The roughly 6.22 Wh test reduction is the largest adjacent difference in Table 1. This change supplies both a different input representation and additional first-layer weights.

Increasing the requested resolution to 32, 128, and 256 bins lowers mean validation RMSE to 71.79, 71.27, and 70.75 Wh. Run 9 has the lowest mean among all ten candidates and is selected as the final model. Compared with Run 8, its three validation-RMSE changes are approximately −1.04, +0.02, and −0.53 Wh. The mean preference is modest and is not an improvement on every partition. The bin-count increases are also unequal refinement steps.

At 512 bins, Run 10 lowers test RMSE by about 0.6 Wh, but mean validation RMSE rises by about 0.63 Wh and test MAE rises by 0.28 Wh. All three validation scores are worse than Run 9's. Parameter count rises from 1,312,257 to 2,185,473, an increase of 66.54%. Retaining Run 9 follows the same validation criterion used for the other candidates; it does not rely on choosing the lowest test RMSE.

Table 2 Results on the reference test partition. Published values are quoted from Candanedo et al. [1, Table 5] at their printed precision. Run 9 is the selected model.

Model	RMSE (Wh)	MAE (Wh)	R^2
Reference: MLR	93.18	51.97	0.16
Reference: radial SVM	70.74	31.36	0.52
Reference: random forest	68.48	31.85	0.54†
Reference: GBM	66.65	35.22	0.57
Run 1: 64-unit baseline	82.22	45.15	0.345
Run 5: standard NN	73.01	35.13	0.484
Run 9: final 256-bin NN	64.05	28.37	0.603
Run 10: 512-bin NN	63.45	28.65	0.610

† Using the reported test-target variance of about 10,323 Wh^2, RMSE 68.48 Wh implies $R^2 \approx 0.546$ by Eq. (5), whereas the quoted source value is 0.54. The quoted value is retained; individual reference predictions are unavailable to resolve the difference.

C. Analysis

1. Comparison with the reference study

Table 2 compares the NN results with the published reference scores. Run 9's test RMSE is about 2.60 Wh lower than the GBM value, a 3.9% reduction. Its MAE is about 6.85 Wh lower, a 19.5% reduction. Its test RMSE is also about 22.1% below Run 1 and 12.3% below Run 5. These are lower reported errors on the reference partition, subject to the evaluation limits discussed below. They do not establish a statistically reliable advantage over GBM.

For fixed evaluation targets, Eq. (5) gives $R^2 = 1 - \mathrm{RMSE}^2/s^2_{y,\mathrm{eval}}$, where $s^2_{y,\mathrm{eval}} = n^{-1}\sum_i (y_i - \overline{y}_{\mathrm{eval}})^2$. RMSE and R^2 therefore summarize the same squared errors. Run 9's $R^2 = 0.603$ means its squared error is about 39.7% of that obtained by predicting the test mean. It does not identify the physical causes of appliance use. MAE gives a different view: Run 10 lowers RMSE while increasing average absolute error.

2. Validation–test gap

Mean validation RMSE exceeds test RMSE by about 0.18 Wh in Run 1, 2.29 Wh in Run 5, 6.71 Wh in Run 9, and 7.93 Wh in Run 10. From Run 5 to Run 9, test error falls by about 8.96 Wh, compared with 4.55 Wh in mean validation error. The two evaluations therefore show different sizes of improvement.

One possible explanation is training-set size. Each validation fit uses 11,842 rows, while the final fit uses 14,803, about 25% more. The fits also differ in scaling, bin edges, number of batch updates, and the observations being evaluated. Development-time test inspection and the shared schedule-selection pool are further limitations. The available experiments do not separate these effects. The validation trend supports the expanded configuration, but the larger test gain should not be assigned entirely to the representation change.

3. Model size and remaining errors

The ramps may make changes across individual input ranges easier for the network to represent. However, Run 9 also has about 31 times Run 5's parameter count and a different selected schedule. The observed gain belongs to the combined feature-plus-network configuration. A size-matched standard network would address the specific question of how much of the gain comes from the representation.

The final mean-validation differences are small relative to the roughly 0.8–1.1 Wh split standard deviations of the piecewise models. Those deviations describe partition-to-partition variation, not uncertainty in paired model differences. With three overlapping partitions and one initialization, the scores support descriptive selection rather than a significance claim. The higher mean validation error at 512 bins justifies stopping at 256 bins within the tested set; it does not show that a globally best architecture has been found.

High-energy observations remain difficult (Fig. 3). The training-derived 95th-percentile threshold is 330 Wh. The Run 9 error summary gives 232 test observations at or above this level, or 4.70% of the test set, contributing 60.61% of squared error. Group RMSE is 229.90 Wh above the threshold and 41.17 Wh below it. Overall prediction-minus-

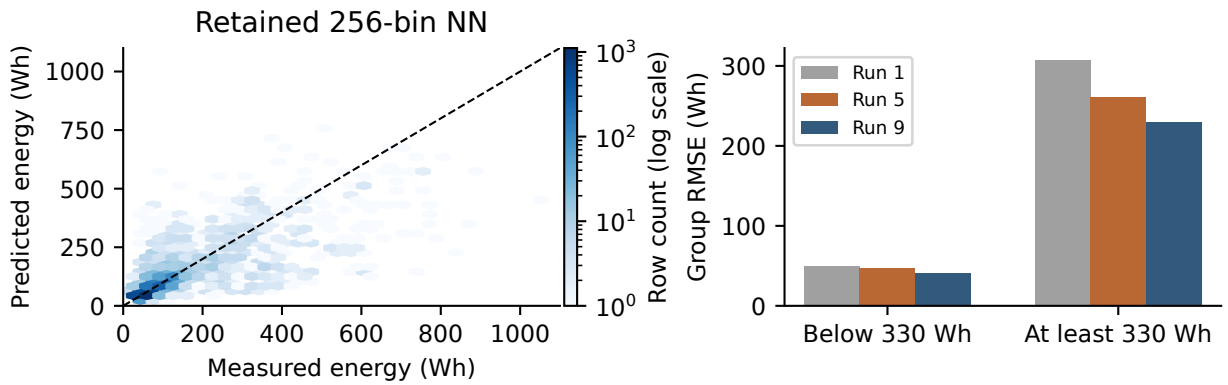


Fig. 3 Test predictions and grouped errors. The left panel shows Run 9 with observation counts and a perfect-agreement line. The right compares Runs 1, 5, and 9 below and at or above the training-derived 330 Wh threshold.

measurement bias is −3.41 Wh. A small overall bias therefore coexists with substantial peak errors. Occupancy or appliance-state measurements could help investigate these peaks, but that hypothesis has not been tested.

4. Scope of the evidence

The data cover one house and part of one year. Nearby observations are related, and hourly weather has been interpolated. Random partitions therefore test a narrower task than prediction in a later period or another house [21]. The schedule-selection dependence, one initialization, and repeated test inspection further limit generalization claims [17]. Reference models use different preprocessing and tuning procedures, and their individual predictions are unavailable here. The comparison consequently supports an exploratory NN result on this benchmark, while leaving statistical significance, physical causality, future forecasting, and transfer to other houses unexplored.

V. Conclusions

In this exploratory comparison, a neural network with 256-bin piecewise-linear inputs has the lowest mean validation RMSE among ten tested configurations. The standard network comparisons reduce test RMSE from 82.22 to 73.01 Wh; the selected expanded network reaches 64.05 Wh, with MAE of 28.37 Wh and $R^2 = 0.603$. The reference GBM reports 66.65 Wh, 35.22 Wh, and 0.57.

The 512-bin candidate has slightly lower test RMSE but higher validation error and MAE, with 66.54% more parameters. Run 9 is therefore retained under the stated validation criterion. Its gains cannot be attributed to representation alone because the expansion also changes parameter count and the selected training schedule.

A useful follow-up would compare equally sized standard and expanded networks, choose schedules within each outer fitting subset, and evaluate the selected procedure on a new temporal holdout. Several initialization seeds would help assess training variability. Investigation of high-energy intervals should precede claims about reliable peak prediction, and data from other houses would be needed to assess transfer beyond this benchmark.

References

[1] Candanedo, L. M., Feldheim, V., and Deramaix, D., "Data driven prediction models of energy use of appliances in a low-energy house," *Energy and Buildings*, Vol. 140, 2017, pp. 81–97. doi:10.1016/j.enbuild.2017.01.083.

[2] Goodfellow, I., Bengio, Y., and Courville, A., *Deep Learning*, MIT Press, 2016. URL https://www.deeplearningbook.org/, chapters 6–8; author-hosted text.

[3] Rumelhart, D. E., Hinton, G. E., and Williams, R. J., "Learning representations by back-propagating errors," *Nature*, Vol. 323, 1986, pp. 533–536. doi:10.1038/323533a0, URL https://www.cs.toronto.edu/~hinton/absps/naturebp.pdf.

[4] Candanedo, L. M., "Appliances-energy-prediction-data," Author's public data and code repository, n.d. URL https://github.com/LuisM78/Appliances-energy-prediction-data, original training.csv and testing.csv; accessed October 30, 2026.

[5] Candanedo, L., "Appliances Energy Prediction," UCI Machine Learning Repository, dataset, 2017. doi:10.24432/C5VC8G, URL https://archive.ics.uci.edu/dataset/374/appliances+energy+prediction, cC BY 4.0; accessed October 30, 2026.

[6] Hastie, T., Tibshirani, R., and Friedman, J., *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed., Springer, New York, 2009. doi:10.1007/978-0-387-84858-7, chapters 5, 7, and 10.

[7] scikit-learn developers, "OneHotEncoder," scikit-learn 1.9 documentation, n.d. URL https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html, accessed October 30, 2026.

[8] scikit-learn developers, "StandardScaler," scikit-learn 1.9 documentation, n.d. URL https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html, accessed October 30, 2026.

[9] LeCun, Y., Bottou, L., Orr, G. B., and Müller, K.-R., "Efficient Backprop," *Neural Networks: Tricks of the Trade*, Lecture Notes in Computer Science, Vol. 1524, Springer, 1998. URL https://leon.bottou.org/papers/lecun-98x, section 4.3: normalizing the inputs.

[10] scikit-learn developers, "Common pitfalls and recommended practices," scikit-learn 1.9 documentation, n.d. URL https://scikit-learn.org/stable/common_pitfalls.html, sections 12.1–12.2, preprocessing and data leakage; accessed October 30, 2026.

[11] Glorot, X., Bordes, A., and Bengio, Y., "Deep Sparse Rectifier Neural Networks," *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, Proceedings of Machine Learning Research, Vol. 15, PMLR, 2011, pp. 315–323. URL https://proceedings.mlr.press/v15/glorot11a.html.

[12] scikit-learn developers, "Metrics and scoring: quantifying the quality of predictions," scikit-learn 1.9 documentation, n.d. URL https://scikit-learn.org/stable/modules/model_evaluation.html, section 3.4.6, regression metrics; accessed October 30, 2026.

[13] Kingma, D. P., and Ba, J., "Adam: A Method for Stochastic Optimization," *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1412.6980.

[14] PyTorch contributors, "Adam," PyTorch 2.14 documentation, n.d. URL https://docs.pytorch.org/docs/2.14/generated/torch.optim.Adam.html, algorithm, defaults, and coupled weight decay; accessed October 30, 2026.

[15] PyTorch contributors, "ReduceLROnPlateau," PyTorch 2.14 documentation, n.d. URL https://docs.pytorch.org/docs/2.14/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html, accessed October 30, 2026.

[16] scikit-learn developers, "train_test_split," scikit-learn 1.9 documentation, n.d. URL https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html, accessed October 30, 2026.

[17] Cawley, G. C., and Talbot, N. L. C., "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation," *Journal of Machine Learning Research*, Vol. 11, No. 70, 2010, pp. 2079–2107. URL https://jmlr.org/papers/v11/cawley10a.html.

[18] PyTorch contributors, "Reproducibility," PyTorch 2.14 documentation, n.d. URL https://docs.pytorch.org/docs/2.14/notes/randomness.html, accessed October 30, 2026.

[19] Gorishniy, Y., Rubachev, I., and Babenko, A., "On Embeddings for Numerical Features in Tabular Deep Learning," *Advances in Neural Information Processing Systems*, Vol. 35, 2022, pp. 24991–25004. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/9e9f0ffc3d836836ca96cbf8fe14b105-Abstract-Conference.html.

[20] NumPy developers, "numpy.quantile," NumPy 2.5 documentation, n.d. URL https://numpy.org/doc/stable/reference/generated/numpy.quantile.html, linear quantile interpolation; accessed October 30, 2026.

[21] Roberts, D. R., Bahn, V., Ciuti, S., Boyce, M. S., Elith, J., Guillera-Arroita, G., Hauenstein, S., Lahoz-Monfort, J. J., Schröder, B., Thuiller, W., Warton, D. I., Wintle, B. A., Hartig, F., and Dormann, C. F., "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, Vol. 40, No. 8, 2017, pp. 913–929. doi:10.1111/ecog.02881.